

KOGNIX Enterprise AI Platform

Production-Grade Air-Gapped Kubernetes Reference Architecture

Document Type: Solution Architecture

Deployment Model: On-Premises / Private Cloud / Fully Air-Gapped

Target Environment: Regulated Enterprises, Government, Defense, Healthcare, Finance, Critical Infrastructure

Architecture Pattern: Kubernetes-Based, Multi-Zone, Highly Available Enterprise AI Platform

1. Executive Summary

The KOGNIX Enterprise AI Platform is a sovereign, production-grade generative AI architecture designed for organizations that cannot permit enterprise data, intellectual property, regulated information, prompts, documents, or model interactions to leave their controlled security boundary.

The platform combines three core capabilities into a cohesive enterprise AI stack:

- **KOGNIX AI STUDIO** — application, workflow, RAG, agent and orchestration layer.
- **KOGNIX Intelligence** — local AI model serving and GPU-accelerated inference layer.
- **KOGNIX Analytics** — end-to-end observability, evaluation, tracing, prompt management and audit layer.

The architecture is designed to operate with **zero runtime Internet connectivity**. All models, documents, prompts, embeddings, vector data, traces, logs, application data and operational telemetry remain within the customer-controlled environment. The runtime platform has no dependency on external LLM APIs, public model registries, SaaS telemetry endpoints or external DNS services.

The resulting platform provides an enterprise-controlled AI environment in which applications are created and orchestrated through KOGNIX AI STUDIO, inference is executed locally by KOGNIX Intelligence, and the complete AI lifecycle is observable through KOGNIX Analytics.

2. Product Architecture

The KOGNIX platform is organized into three logical product layers.

Product	Architectural Role	Primary Responsibility
KOGNIX AI STUDIO	Application & Workflow Layer	Applications, workflows, agents, RAG, prompts and APIs
KOGNIX Intelligence	AI Inference & Model Serving Layer	Local LLM/model serving and GPU inference
KOGNIX Analytics	Observability & Evaluation Layer	Tracing, telemetry, evaluation, dashboards and audit

2.1 KOGNIX AI STUDIO

KOGNIX AI STUDIO provides the enterprise application and orchestration layer.

Its primary capabilities include:

- Low-code/no-code AI application development.
- Visual workflow and DAG orchestration.
- Agent and tool orchestration.
- RAG pipeline construction.
- Document ingestion.
- Chunking and retrieval orchestration.
- Embedding coordination.
- Prompt management and versioning.
- Experimentation and testing.
- Multi-tenant workspaces.
- Project and role management.
- API exposure for enterprise applications.

KOGNIX AI STUDIO is the principal interaction point between enterprise users/applications and the AI platform.

2.2 KOGNIX Intelligence

KOGNIX Intelligence is the local inference and model-serving layer.

It provides:

- On-premises LLM inference.
- Multi-model serving.
- GPU-accelerated inference.
- Concurrent request handling.
- Batching/continuous-batching capable inference runtimes.
- Local model catalog.
- Model metadata and versioning.
- Model-serving APIs.
- Embedding-model serving.
- Optional multimodal model serving.
- Health and performance monitoring.

The architecture supports multiple open-weight models such as Llama, Mistral and Qwen, subject to the customer's approved model inventory and licensing requirements.

2.3 KOGNIX Analytics

KOGNIX Analytics provides centralized observability and evaluation.

It captures:

- End-to-end traces.
- Workflow spans.
- LLM observations.
- Retrieval operations.
- Prompt and response telemetry.
- Latency.
- Token utilization.
- Cost metrics.
- Error rates.
- Success rates.
- RAG evaluation metrics.
- Agent evaluation.
- Prompt experiments.
- Audit events.

KOGNIX Analytics is intended to provide engineering, product, security and compliance teams with a common operational view of the AI environment.

3. Architecture Principles

The reference architecture follows the following principles.

3.1 Sovereignty by Design

All enterprise data remains within the customer's infrastructure.

3.2 Runtime Air-Gap

The production AI environment has no outbound Internet connectivity.

3.3 Local Model Serving

Models are imported into the controlled environment and served locally.

3.4 Zero External AI Dependency

No production workflow depends on:

- External LLM APIs.
- Public model endpoints.
- SaaS inference.
- External telemetry.
- Public model registries.

3.5 Defense in Depth

Security is implemented through multiple independent controls:

- Network segmentation.
- Kubernetes namespaces.
- Network policies.
- Firewall enforcement.
- Identity and RBAC.
- TLS.
- Encryption at rest.
- Signed artifacts.
- Audit logging.
- SIEM integration.

3.6 Separation of Concerns

Application orchestration, inference, analytics, data and infrastructure management are separated into distinct architectural zones.

3.7 Highly Available by Default

Production deployments should avoid single-node dependencies for critical services.

4. High-Level Logical Architecture

The platform should be represented as a series of security and trust zones enclosed within an explicit:

AIR-GAP SECURITY BOUNDARY — NO INTERNET / NO EXTERNAL NETWORK

boundary.

The logical diagram should contain the following zones.

4.1 User Access Zone

This is the enterprise user interaction layer.

Components:

- Corporate desktops.
- Internal browsers.
- Internal applications.
- Enterprise portals.
- Internal API consumers.
- Administrative workstations.

Users authenticate through the customer's internal identity infrastructure.

Typical flow:

User → Internal Load Balancer → KOGNIX AI STUDIO

5. Application & API Zone

This zone hosts KOGNIX AI STUDIO.

Components

- KOGNIX AI STUDIO Web Frontend.

- API services.
- Workflow engine.
- RAG orchestration services.
- Application runtime.
- Data ingestion services.
- Authentication integration.
- Internal API gateway.
- Internal reverse proxy/load balancer.

Responsibilities

KOGNIX AI STUDIO:

1. Receives user/application requests.
 2. Authenticates and authorizes the request.
 3. Selects the appropriate application/workflow.
 4. Executes workflow logic.
 5. Performs retrieval orchestration.
 6. Invokes embedding services.
 7. Invokes KOGNIX Intelligence.
 8. Receives model responses.
 9. Applies workflow/business logic.
 10. Returns the response.
 11. Emits telemetry to KOGNIX Analytics.
-

6. AI & Inference Zone

The AI & Inference Zone contains KOGNIX Intelligence.

Components

- KOGNIX Intelligence API.
- Model controller.
- Inference runtime.
- GPU scheduling layer.
- Model registry.
- Model artifacts.
- Embedding-model services.
- Optional vision/speech services.
- GPU node pool.

GPU Infrastructure

The platform may use:

- NVIDIA GPU servers.
- NVIDIA HGX systems.
- NVIDIA DGX-class infrastructure.
- Enterprise GPU clusters.

The exact GPU configuration should be selected based on:

- Model parameter count.
 - Quantization.
 - Context window.
 - Concurrent users.
 - Requests per second.
 - Token generation rate.
 - Availability requirements.
-

7. Observability & Analytics Zone

KOGNIX Analytics is deployed independently from the application and inference workloads.

Components

- KOGNIX Analytics Web UI.
- Analytics API.
- Background workers.
- Trace ingestion.
- Evaluation workers.
- Event storage.
- Metadata database.
- Optional cache.
- Analytics dashboards.

Primary Data

KOGNIX Analytics receives:

- Request traces.
- Workflow spans.

- LLM calls.
 - Retrieval events.
 - Prompt versions.
 - Model identifiers.
 - Latency.
 - Token counts.
 - Errors.
 - Evaluation results.
 - User/project identifiers.
 - Audit events.
-

8. Data & Storage Zone

The Data & Storage Zone contains persistent enterprise AI data.

Core storage components

PostgreSQL

Used for transactional metadata such as:

- Application configuration.
- Users.
- Projects.
- API keys.
- Workflow metadata.
- Configuration.

Vector Database

Used for:

- Document embeddings.
- Semantic retrieval.
- RAG indexes.
- Knowledge bases.

Object Storage

MinIO or equivalent may be used for:

- Documents.

- Uploaded files.
- Model artifacts.
- Workflow artifacts.
- Backups.
- Evaluation datasets.

Analytics Event Store

A ClickHouse-like analytics datastore may be used for high-volume telemetry and event storage.

Central Log Store

Collects:

- Application logs.
- Kubernetes logs.
- Security logs.
- Audit events.
- Infrastructure logs.

The source architecture explicitly identifies block storage for PostgreSQL, ClickHouse-like systems and vector databases, with object storage for documents, models and backups.

9. Management & Operations Zone

This zone provides platform administration and security operations.

Components:

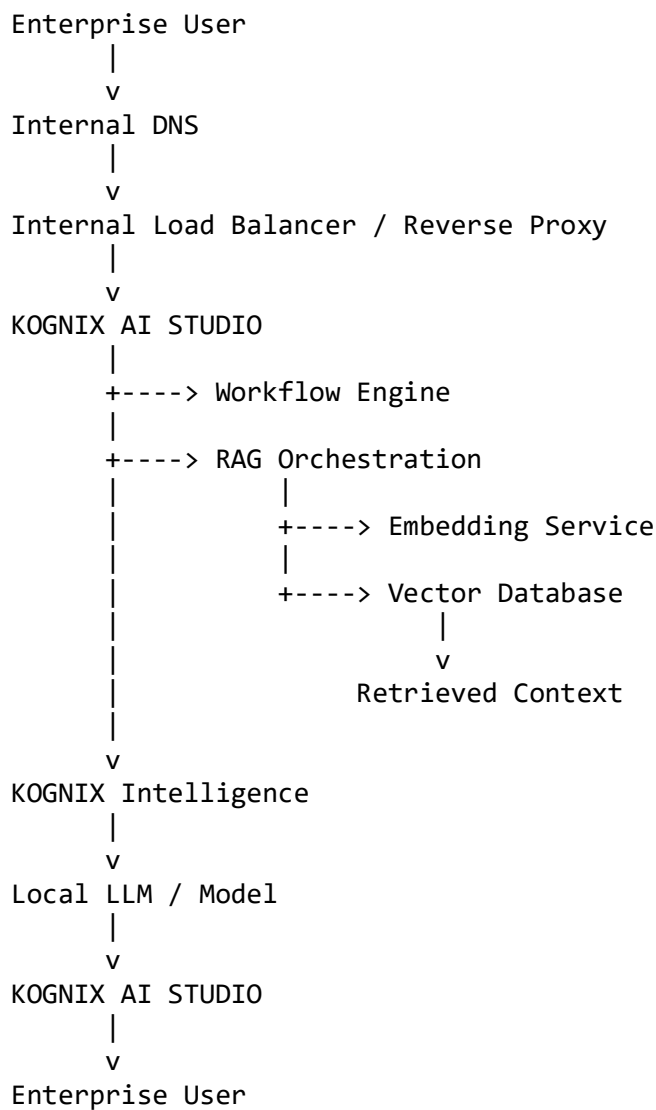
- Kubernetes management.
- Monitoring.
- Alerting.
- Central logging.
- Internal SIEM integration.
- Backup management.
- Artifact repository.
- Internal container registry.
- Model registry.
- Vulnerability scanning.
- Certificate management.

- KMS/HSM integration.
- Offline update station interface.

The Management Zone must itself remain within the customer’s security architecture.

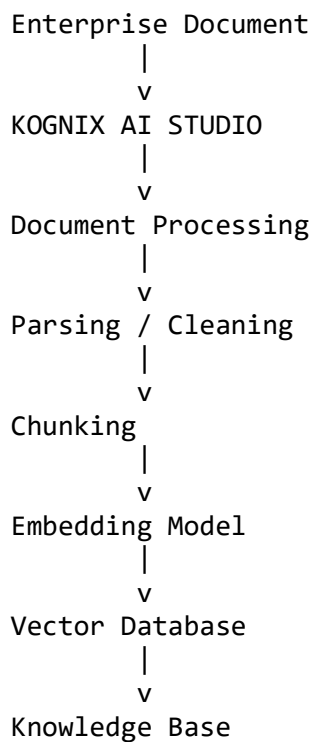
10. End-to-End Data Flows

10.1 User Query Flow



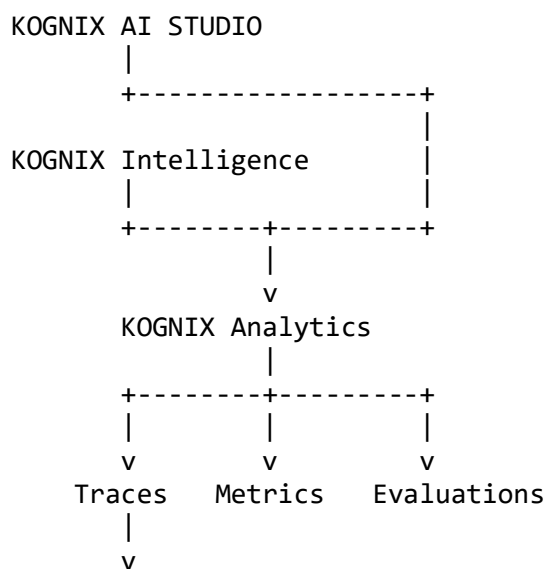
At each major stage, telemetry is emitted to KOGNIX Analytics.

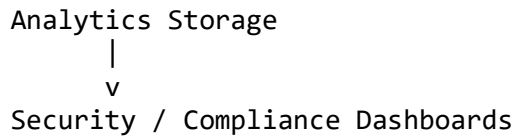
11. Document Ingestion Flow



The original document may also be retained in object storage subject to the enterprise retention policy.

12. Observability Flow





The architecture requires all major components to emit traces, spans and observations to KOGNIX Analytics.

13. Explicitly Prohibited Runtime Connections

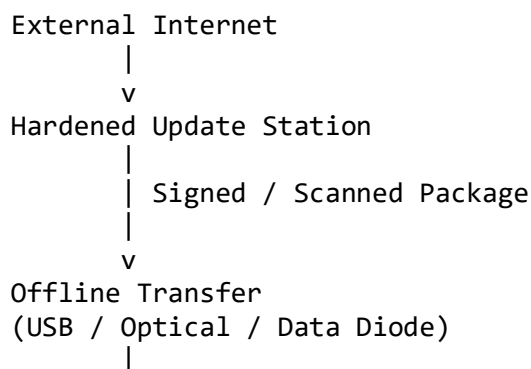
The production environment must have **no runtime connections** to:

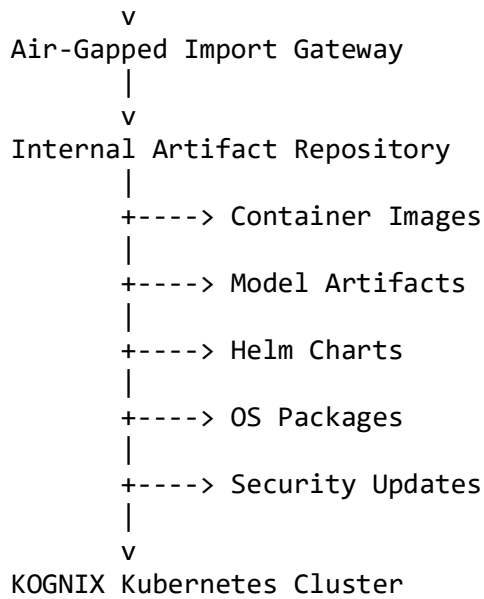
- OpenAI APIs.
- Google Gemini APIs.
- Anthropic APIs.
- Public cloud inference APIs.
- Hugging Face model endpoints.
- Public model registries.
- External telemetry services.
- External DNS.
- Public package repositories.
- Internet-based update services.

If external information is required, it must be imported through the controlled offline update process.

14. Air-Gapped Update Architecture

The architecture includes a controlled update path.





The supplied architecture specifically calls for a trusted update station, offline signed transfer and an internal repository such as Harbor or Nexus.

Update validation

Every package should undergo:

1. Source verification.
2. Malware scanning.
3. Vulnerability scanning.
4. Cryptographic signature validation.
5. SBOM validation where available.
6. License validation.
7. Model provenance verification.
8. Compatibility testing.
9. Offline staging.
10. Production promotion.

15. Physical / Deployment Architecture

15.1 Kubernetes Platform

The recommended deployment foundation is Kubernetes.

Supported architectural choices include:

- RKE2.
- Red Hat OpenShift.
- Enterprise Kubernetes distributions.
- Vanilla Kubernetes where operationally appropriate.

The source requirements explicitly identify Kubernetes as the base platform and permit RKE2, OpenShift or vanilla Kubernetes.

16. Kubernetes Cluster Topology

A production cluster should be divided into logical node pools.

16.1 Control Plane

Recommended:

- 3 control-plane nodes minimum.
- Dedicated control-plane resources.
- etcd distributed across the control plane.
- Anti-affinity across physical hosts/racks.

Responsibilities:

- Kubernetes API.
 - Scheduler.
 - Controller manager.
 - etcd.
-

16.2 Application Node Pool

Hosts:

- KOGNIX AI STUDIO.
- KOGNIX Analytics.
- API gateways.
- Internal services.

Recommended characteristics:

- 3+ nodes.
- SSD-backed local/cache storage where required.

- CPU and memory optimized.
 - Kubernetes anti-affinity.
-

16.3 GPU Node Pool

Hosts:

- KOGNIX Intelligence.
- LLM inference.
- Embedding models.
- Optional multimodal models.

GPU nodes should be isolated from general application workloads.

Recommended controls:

- NVIDIA device plugin/operator.
 - GPU resource quotas.
 - Node labels.
 - Taints/tolerations.
 - GPU-aware scheduling.
 - Pod anti-affinity where required.
-

16.4 Storage Node Pool

Hosts:

- Object storage.
- Database workloads where Kubernetes-native databases are used.
- Vector database.
- Analytics storage.

Storage should use redundant enterprise storage rather than relying on ephemeral container disks.

16.5 Operations Node Pool

Hosts:

- Monitoring.
- Logging.

- Backup agents.
 - Security tooling.
 - Vulnerability scanners.
 - Internal operational services.
-

17. Kubernetes Namespace Architecture

A recommended namespace structure is:

```
kognix-platform
|
+-- kognix-ai-studio
|
+-- kognix-intelligence
|
+-- kognix-analytics
|
+-- kognix-data
|
+-- kognix-observability
|
+-- kognix-security
|
+-- kognix-ops
|
+-- kognix-registry
```

Namespace responsibilities

kognix-ai-studio

- Frontend.
- API.
- Workflow engine.
- RAG services.

kognix-intelligence

- Model-serving control plane.
- Model runtimes.
- Inference services.

kognix-analytics

- Analytics UI.

- Analytics API.
- Workers.

kognix-data

- Database and storage integrations.

kognix-observability

- Metrics.
- Logging.
- Alerting.

kognix-security

- Security agents.
- Certificate management.
- Policy enforcement.

kognix-registry

- Internal artifact/model registry.
-

18. Reference Network Segmentation

The physical architecture should use VLAN and firewall segmentation.

A representative implementation can use:

Zone	Example VLAN	Example Subnet	Purpose
User	110	10.110.0.0/24	Enterprise users
Application	120	10.120.0.0/24	AI STUDIO
Inference	130	10.130.0.0/24	GPU workloads
Analytics	140	10.140.0.0/24	Analytics
Data	150	10.150.0.0/24	DB/vector/object storage
Management	160	10.160.0.0/24	Platform management
Security	170	10.170.0.0/24	SIEM/security tooling
Backup	180	10.180.0.0/24	Backup infrastructure
Registry	190	10.190.0.0/24	Internal artifacts

These CIDRs are **reference values**, not mandatory customer addressing. They should be replaced with the customer's IP addressing plan.

19. Firewall Policy

The firewall architecture should follow a default-deny model.

Representative permitted flows:

Source	Destination	Protocol	Purpose
User Zone	AI STUDIO LB	HTTPS	User access
AI STUDIO	Intelligence	HTTPS	LLM inference
AI STUDIO	Vector DB	TCP	Retrieval
AI STUDIO	PostgreSQL	TCP	Metadata
AI STUDIO	Analytics	HTTPS	Telemetry
Intelligence	Model Registry	HTTPS	Model retrieval
Analytics	Analytics DB	TCP	Event storage
Platform	Internal DNS	DNS	Name resolution
Platform	SIEM	Syslog/HTTPS	Security logging
Backup	Data Zone	Backup protocol	Backup
Admin	Management	SSH/HTTPS	Administration

No application or GPU workload should have a default route to the Internet.

20. Internal DNS

The production environment must not depend on public DNS.

Use:

- Internal DNS.
- Kubernetes CoreDNS.
- Enterprise DNS forwarding only to approved internal zones.

Example internal service names:

```
ai-studio.kognix.internal
intelligence.kognix.internal
analytics.kognix.internal
postgres.kognix.internal
vector.kognix.internal
registry.kognix.internal
```

21. High Availability Architecture

21.1 Application HA

KOGNIX AI STUDIO should run multiple replicas.

Example:

```
AI STUDIO API
|
+-- Replica 1
+-- Replica 2
+-- Replica 3
```

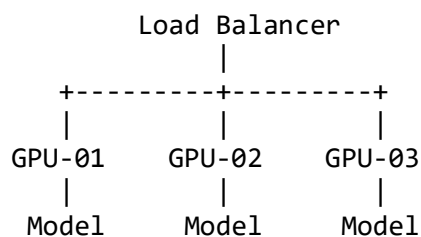
Use:

- Kubernetes Deployments.
- Pod anti-affinity.
- PodDisruptionBudgets.
- Readiness probes.
- Liveness probes.
- Horizontal scaling where appropriate.

21.2 Intelligence HA

KOGNIX Intelligence should use multiple inference nodes where availability requirements justify it.

Possible architecture:



Models may be replicated according to demand and GPU capacity.

21.3 Analytics HA

Analytics services should use multiple application replicas.

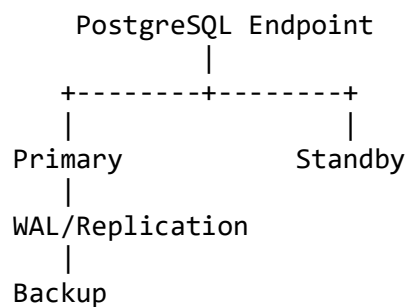
Persistent analytics storage should be replicated independently.

22. Database Architecture

PostgreSQL

PostgreSQL stores transactional metadata.

Recommended production topology:



High availability should be implemented using an enterprise-supported PostgreSQL HA pattern.

23. Vector Database Architecture

The vector layer stores embeddings for RAG.

Production design should include:

- Multiple replicas where supported.
- Persistent storage.
- Backup.
- Index rebuild procedures.
- Capacity monitoring.
- Tenant isolation.
- Encryption.
- Access controls.

The vector database should never be directly accessible from user networks.

Only authorized KOGNIX AI STUDIO services should access it.

24. Object Storage Architecture

MinIO or equivalent enterprise object storage can provide:

- Document storage.
- Model artifact storage.
- Backup storage.
- Evaluation datasets.
- Workflow artifacts.

Production storage should use redundancy/erasure coding where supported.

The supplied architecture explicitly calls for object storage such as MinIO or equivalent and object-store replication or erasure coding.

25. Model Registry

The internal model registry is the authoritative source for approved models.

Each model artifact should have:

- Model name.
- Version.
- Architecture.
- Parameter count.
- Quantization.
- SHA-256 checksum.
- License.
- Source/provenance.
- Security scan status.
- Approval status.
- Intended workload.
- GPU requirements.

Example:

```
/models/  
  qwen/  
    qwen-72b/  
      v1/  
  llama/  
    llama-70b/  
      v1/  
  mistral/  
    mistral-large/  
      v1/
```

Only approved artifacts may be promoted into production inference.

26. Security Architecture

26.1 Identity

KOGNIX should integrate with enterprise identity infrastructure.

Supported enterprise patterns include:

- Active Directory.
- LDAP.
- SAML.
- OIDC.

The supplied requirements explicitly call for integration with internal IdP infrastructure and RBAC/ABAC.

26.2 Authorization

Authorization should operate at multiple levels:

```
Enterprise  
|  
Workspace  
|  
Project  
|  
Application  
|  
Workflow  
|  
API Key
```

Roles can include:

- Platform Administrator.
 - Security Administrator.
 - Workspace Administrator.
 - AI Engineer.
 - Developer.
 - Analyst.
 - Auditor.
 - Read Only.
-

27. Service-to-Service Authentication

System-to-system communication should use:

- Service accounts.
- Short-lived credentials where supported.
- API keys.
- Mutual TLS where required.
- Kubernetes service identities.

Credentials must never be embedded into container images.

Secrets should be stored in:

- Kubernetes Secrets protected by encryption.
 - Enterprise secrets management.
 - KMS/HSM-backed infrastructure where available.
-

28. Encryption

Encryption in Transit

All sensitive service communication should use TLS.

Recommended:

User
|
TLS
|

AI STUDIO
|
TLS
|
Intelligence

The same principle applies to:

- Databases.
- Object storage.
- Vector databases.
- Analytics.
- Registry.
- SIEM.

The source requirements explicitly specify TLS for internal service-to-service communication.

Encryption at Rest

Encryption should cover:

- PostgreSQL.
- Vector databases.
- Analytics storage.
- Object storage.
- Model artifacts.
- Backup repositories.
- Logs.

Customer-managed keys should be used where enterprise KMS/HSM infrastructure is available.

29. Container Security

All production container images should undergo:

1. Source verification.
2. SBOM generation.
3. Vulnerability scanning.
4. Malware scanning.

5. License analysis.
6. Image signing.
7. Signature verification during deployment.

The production cluster should accept only approved images.

The supplied requirements specifically call for signed images, vulnerability scanning before air-gap import and minimal base images.

30. Kubernetes Security

Recommended controls:

- NetworkPolicies.
 - Pod Security Standards.
 - Namespace isolation.
 - RBAC.
 - Resource quotas.
 - Limit ranges.
 - Admission policies.
 - Image signature verification.
 - Secrets encryption.
 - Node hardening.
 - Restricted privileged workloads.
 - Read-only root filesystems where compatible.
 - Non-root containers where supported.
-

31. Network Micro-Segmentation

Network policies should enforce application-specific communication.

For example:

```
AI STUDIO
ALLOW -> Intelligence
ALLOW -> PostgreSQL
ALLOW -> Vector DB
ALLOW -> Analytics
DENY   -> Internet
```

Intelligence

ALLOW -> Model Registry
ALLOW -> Analytics
DENY -> User Network except approved API path
DENY -> Internet

Analytics

ALLOW -> Analytics DB
ALLOW -> PostgreSQL
ALLOW -> SIEM
DENY -> Internet

32. Audit & Logging

The platform should capture:

User Events

- Login.
- Logout.
- Authentication failure.
- Application creation.
- Workflow modification.
- Permission changes.
- API key creation.
- API key revocation.

AI Events

- Prompt.
- Model.
- Model version.
- Retrieval operation.
- Retrieved documents.
- Response.
- Token usage.
- Latency.
- Errors.

Administrative Events

- Configuration changes.
- Model deployment.

- Model removal.
- Software update.
- Kubernetes administrative activity.

The source requirements explicitly call for logging prompts, responses, retrieval actions and user actions with timestamps and user IDs.

33. SIEM Integration

KOGNIX logs should be forwarded to the enterprise SIEM.

Possible mechanisms include:

- Syslog.
- HTTPS log ingestion.
- Agent-based forwarding.
- One-way channel.
- Data diode.

The SIEM should remain internal to the customer environment.

34. Compliance Architecture

The architecture is designed to support regulated deployments by providing:

- Data sovereignty.
- Local processing.
- Full auditability.
- Controlled model provenance.
- Controlled software updates.
- Encryption.
- Identity integration.
- Role-based access.
- Traceability.
- Retention controls.
- Internal SIEM integration.

The exact compliance certification or regulatory suitability remains dependent on the customer's implementation, policies, validation procedures and applicable jurisdiction.

35. KOGNIX AI STUDIO — Product Capabilities

Application Development

- Low-code/no-code application builder.
- Internal AI assistants.
- Enterprise chat applications.
- Workflow-driven applications.

Workflow Orchestration

- Visual workflow design.
- DAG execution.
- Conditional routing.
- Multi-step processing.
- Agent workflows.
- Tool invocation.

RAG

- Data source configuration.
- Document ingestion.
- Parsing.
- Chunking.
- Embedding.
- Vector indexing.
- Retrieval.
- Context assembly.
- Generation.

Prompt Engineering

- Prompt library.
- Prompt versioning.
- Testing.
- Experimentation.
- A/B comparison.

Enterprise Integration

- REST APIs.

- Internal enterprise applications.
 - Authentication integration.
 - API keys.
 - Service-to-service integration.
-

36. KOGNIX Intelligence — Product Capabilities

Model Serving

- Multiple local models.
- Model versioning.
- Model selection.
- Model health monitoring.
- API-based inference.

Performance

- GPU acceleration.
- Concurrent requests.
- Batching.
- Continuous batching where supported by the selected runtime.
- GPU resource allocation.

Model Management

- Local model catalog.
- Model metadata.
- Artifact checksums.
- Model approval status.
- Version control.

Additional AI Models

Where required, the platform can expose:

- Embedding models.
- Reranking models.
- Vision models.
- Speech models.

The architecture should treat these as separate production workloads where their resource and security requirements differ.

37. KOGNIX Analytics — Product Capabilities

Observability

- Distributed traces.
- Workflow spans.
- LLM observations.
- Retrieval observations.
- Error tracking.
- Latency analysis.

Metrics

- Token utilization.
- Request volume.
- Latency.
- Error rate.
- Success rate.
- Model utilization.
- Application utilization.

Evaluation

Potential evaluation dimensions include:

- Relevance.
- Faithfulness.
- Hallucination.
- Retrieval quality.
- Answer quality.
- Agent success.
- Groundedness.

Governance

- Project-level access.
- Audit trails.
- Prompt/version tracking.
- Experiment history.
- Security visibility.

38. Example Use Case 1 — Classified Document Q&A

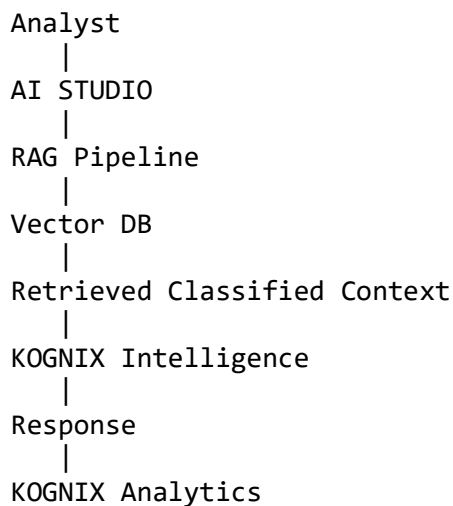
Users

- Defense analysts.
- Intelligence personnel.
- Authorized researchers.

Data

- Classified documents.
- Intelligence reports.
- Internal manuals.
- Historical records.

Architecture



Why Air-Gap Matters

Classified information must not reach public cloud infrastructure or external AI providers.

39. Example Use Case 2 — Government Policy Assistant

Users

- Government officers.
- Policy analysts.

- Compliance teams.

Data

- Internal policies.
- Circulars.
- Government regulations.
- Internal procedures.

KOGNIX AI STUDIO provides the conversational and RAG application.

KOGNIX Intelligence provides local inference.

KOGNIX Analytics provides complete traceability of interactions.

Air-gapping prevents government information from being transmitted outside the controlled environment.

40. Example Use Case 3 — Secure Clinical Decision Support

Users

- Doctors.
- Clinical researchers.
- Hospital staff.

Data

- Hospital protocols.
- Internal clinical guidelines.
- Approved medical documents.
- Patient-related information where authorized.

The platform processes data locally.

No PHI needs to leave the hospital's controlled infrastructure.

KOGNIX Analytics provides traceability for model interactions and retrieval operations.

Any clinical deployment must additionally undergo appropriate clinical safety, validation, privacy and regulatory controls.

41. Example Use Case 4 — Critical Infrastructure Engineering Assistant

Users

- Engineers.
- Maintenance teams.
- Operations personnel.
- Reliability engineers.

Data

- Engineering manuals.
- Equipment documentation.
- Maintenance records.
- SOPs.
- Incident reports.

KOGNIX AI STUDIO provides the engineering knowledge assistant.

KOGNIX Intelligence performs local inference.

KOGNIX Analytics provides operational and model-performance observability.

The air gap prevents operational technology information and sensitive infrastructure documentation from leaving the enterprise environment.

42. Backup Architecture

The backup strategy should cover:

Application

- Configuration.
- Workflow definitions.
- Application metadata.

Database

- PostgreSQL.
- Vector database.
- Analytics metadata.

Object Storage

- Documents.
- Model artifacts.
- Evaluation datasets.

Kubernetes

- Cluster configuration.
- Application manifests.
- Helm charts.
- Secrets recovery material.

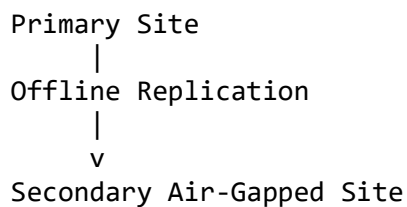
Logs

Retention should follow enterprise regulatory requirements.

43. Disaster Recovery

A secondary air-gapped site can be deployed for disaster recovery.

Possible model:



Replication may include:

- Database backups.
- Object storage.
- Model artifacts.
- Application configuration.
- Kubernetes configuration.

The supplied requirements explicitly identify a secondary air-gapped site as an optional DR architecture.

44. Operational Monitoring

The operations team should monitor:

Kubernetes

- Node health.
- Pod health.
- CPU.
- Memory.
- Storage.
- Network.
- Kubernetes control plane.

GPU

- GPU utilization.
- GPU memory.
- Temperature.
- Power.
- ECC errors.
- Inference throughput.

AI STUDIO

- Request volume.
- Workflow failures.
- Queue depth.
- API latency.

Intelligence

- Model latency.
- Tokens/sec.
- Concurrent requests.
- GPU utilization.
- Model availability.

Analytics

- Trace ingestion rate.
- Storage growth.
- Query performance.
- Evaluation jobs.

45. Capacity Planning

Capacity should be based on workload rather than a fixed hardware prescription.

Primary sizing parameters:

- Number of users.
- Concurrent users.
- Requests per second.
- Average input tokens.
- Average output tokens.
- Context window.
- Model size.
- Quantization.
- Target latency.
- RAG workload.
- Embedding volume.
- Document ingestion rate.
- Retention period.

A production sizing exercise should therefore be performed after collecting actual workload characteristics.

46. Reference GPU Sizing Methodology

A model-serving deployment should estimate:

Required GPU Memory

=

Model Weights

+ KV Cache

+ Runtime Overhead

+ Context Requirements

+ Concurrency Headroom

The number of GPUs should then be derived from:

Required Capacity

/

Capacity per GPU

=

Required GPU Count

with additional redundancy for HA.

For example, a production environment may maintain:

- N GPUs for baseline inference.
- N+1 capacity for node failure.
- Additional capacity for peak workloads.

Exact GPU models and counts should not be fixed until model and concurrency requirements are known.

47. Deployment Sequence

A recommended implementation sequence is:

Phase 1 — Infrastructure

1. Prepare servers.
2. Configure network.
3. Configure VLANs.
4. Configure firewalls.
5. Configure DNS.
6. Prepare storage.
7. Install Kubernetes.

Phase 2 — Security

1. Configure identity.
2. Configure certificates.
3. Configure RBAC.
4. Configure network policies.
5. Configure secrets.
6. Configure image-signing controls.

Phase 3 — Platform

1. Deploy internal registry.
2. Deploy databases.
3. Deploy object storage.
4. Deploy vector database.
5. Deploy observability infrastructure.

Phase 4 — KOGNIX

1. Deploy KOGNIX AI STUDIO.
2. Deploy KOGNIX Intelligence.
3. Import approved models.
4. Deploy KOGNIX Analytics.
5. Configure telemetry.

Phase 5 — Validation

1. Security testing.
2. Network isolation testing.
3. Functional testing.
4. Performance testing.
5. Model evaluation.
6. Disaster recovery testing.

Phase 6 — Production

1. User onboarding.
 2. Application deployment.
 3. Monitoring activation.
 4. Operational handover.
 5. Production acceptance.
-

48. Typical Implementation Timeline

A representative production implementation can be completed in approximately **6–10 weeks after hardware and foundational infrastructure are ready**, depending on:

- Security review.
- Enterprise identity integration.
- Hardware readiness.
- Network approvals.
- Model approval.
- Compliance validation.
- Data onboarding complexity.

The source requirements identify 6–10 weeks as the target deployment timeline.

49. Operational Responsibilities

Platform Team

Responsible for:

- Kubernetes.
- Networking.
- Storage.
- Platform availability.
- Upgrades.

AI Engineering

Responsible for:

- Models.
- Prompt engineering.
- RAG.
- Evaluation.
- AI application development.

Security Team

Responsible for:

- IAM.
- Vulnerability management.
- SIEM.
- Security monitoring.
- Artifact validation.

Compliance Team

Responsible for:

- Retention.
- Audit requirements.
- Validation.
- Regulatory controls.

Operations

Responsible for:

- Monitoring.
- Backup.
- DR.
- Incident response.
- Capacity management.

The supplied architecture identifies platform, AI engineering, security/compliance and operations responsibilities as core operating roles.

50. Air-Gapped Model Update Procedure

A production model update should follow:

Step 1 — Acquire

Obtain the model through an approved external process.

Step 2 — Verify

Verify:

- Source.
- License.
- Checksum.
- Provenance.
- Model integrity.

Step 3 — Scan

Perform:

- Malware scanning.
- Security scanning.
- Artifact inspection.

Step 4 — Stage

Place the artifact on the hardened update station.

Step 5 — Sign

Apply enterprise-approved signing metadata.

Step 6 — Transfer

Transfer through:

- Approved removable media.
- Optical media.
- Data diode.

Step 7 — Import

Import into the internal model registry.

Step 8 — Validate

Perform:

- Functional testing.
- Performance testing.
- Safety testing.
- Evaluation testing.

Step 9 — Approve

Security/AI governance approves production promotion.

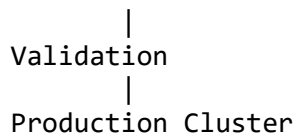
Step 10 — Deploy

KOGNIX Intelligence pulls the approved model from the internal registry.

51. Air-Gapped Software Update Procedure

Software updates follow the same principle:

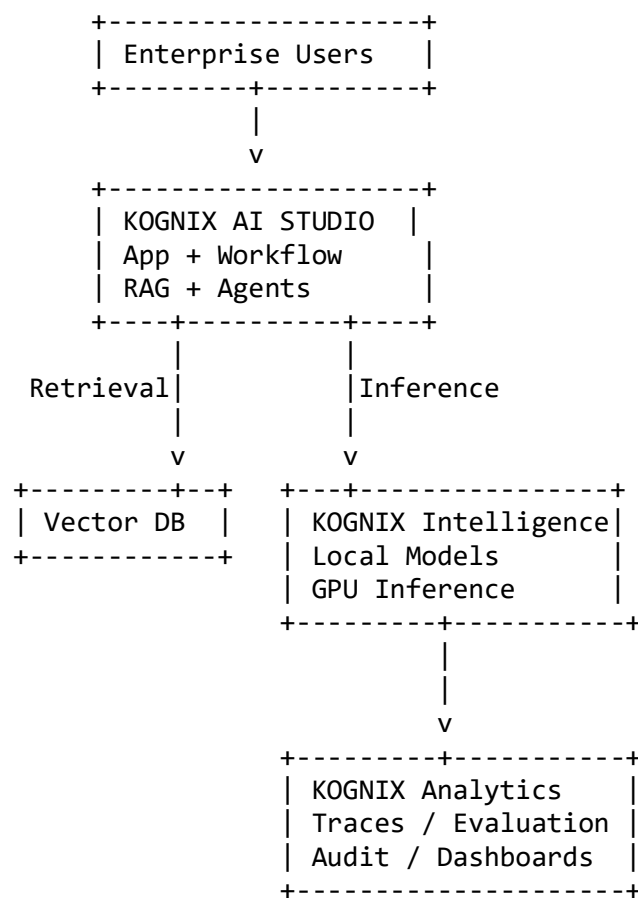
```
Vendor / Upstream
|
External Security Review
|
Hardened Update Station
|
Artifact Verification
|
Offline Transfer
|
Internal Registry
|
Staging Cluster
```



Production must never pull packages directly from the Internet.

52. Component Interaction Model

The principal interaction model is:



53. Trust Boundary Model

The architecture contains several distinct trust boundaries.

Boundary 1 — Enterprise User vs Platform

Identity and authorization controls apply.

Boundary 2 — Application vs AI

KOGNIX AI STUDIO may invoke only approved KOGNIX Intelligence endpoints.

Boundary 3 — AI vs Data

Inference workloads access only approved data services.

Boundary 4 — Platform vs Management

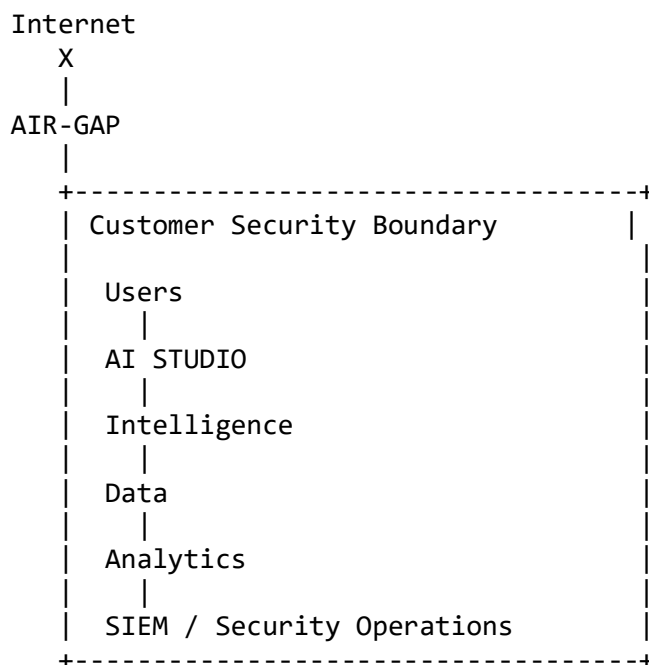
Administrative interfaces are isolated from end-user networks.

Boundary 5 — Air-Gap

No runtime component crosses the external network boundary.

54. Security Posture

The target production posture is:



The key security property is that enterprise AI execution remains inside this boundary.

55. Enterprise Architecture Benefits

Data Sovereignty

Enterprise data remains under customer control.

Reduced Data Egress Risk

Runtime AI processing does not require external AI APIs.

Model Sovereignty

Approved models are locally stored and controlled.

Operational Visibility

AI activity is observable through KOGNIX Analytics.

Enterprise Integration

The platform integrates with internal identity, security, storage and monitoring systems.

Regulatory Support

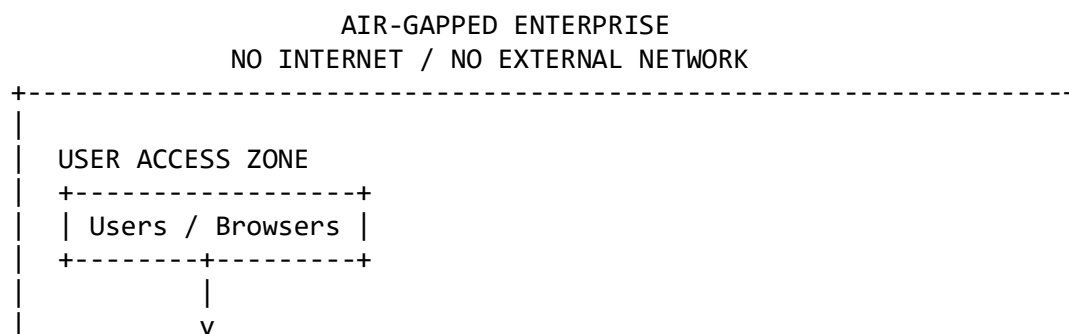
The architecture provides controls required to build a controlled and auditable AI environment.

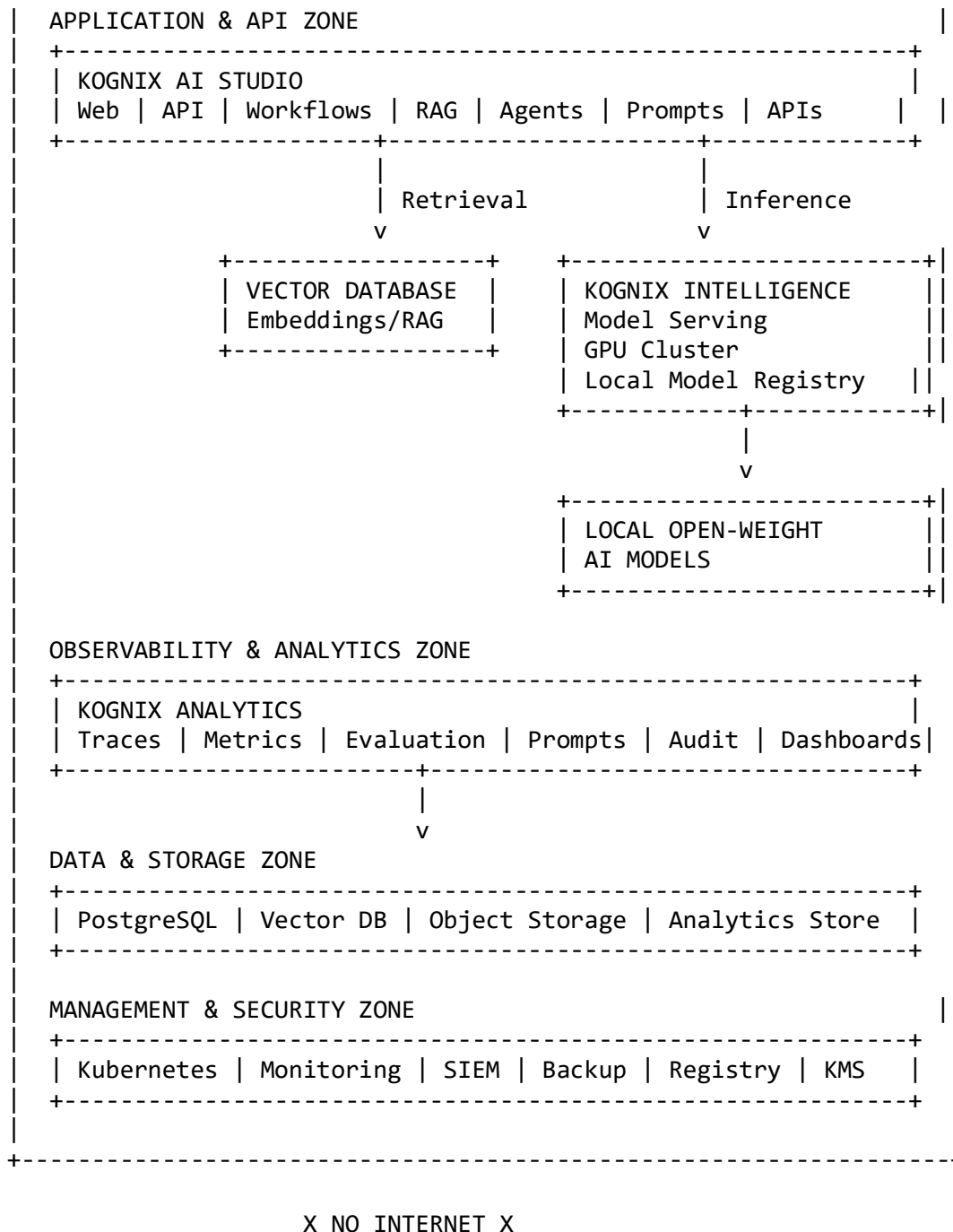
Predictable Infrastructure

Inference infrastructure is owned and operated within the enterprise environment rather than being dependent on external SaaS availability.

56. Reference Architecture Summary

The complete architecture can be summarized as:





57. Final Architecture Position

The KOGNIX Enterprise AI Platform should be deployed as a **sovereign AI infrastructure stack**, rather than as three independent applications.

The architectural relationship is:

KOGNIX AI STUDIO = Build and Orchestrate

KOGNIX Intelligence = Infer and Serve

KOGNIX Analytics = Observe, Evaluate and Govern

Together they provide the application, inference and governance layers required for a production enterprise generative AI environment.

The defining architectural characteristic is that the complete AI lifecycle—from document ingestion and retrieval through model inference, tracing, evaluation, storage and audit—can operate inside the customer’s security boundary without runtime dependence on the public Internet or external AI services.

This architecture therefore provides a foundation for deploying enterprise generative AI in environments where **data sovereignty, network isolation, model control, auditability and operational ownership are mandatory architectural requirements.**